Tài liệu bổ trợ — Bài tập 2

Đặc trưng góc & Phân vùng ảnh

Môn: Thị giác máy tính (121036)

Trường Đại học Giao thông Vận tải TP.HCM

Mục lục

1 Mục tiêu **1**

2 Quy định chung **2**
2.1 Chính sách sử dụng AI . . . 2
2.2 Thư viện . . . 2
2.3 Nộp bài . . . 2

3 Phần A — Harris và hai biến thể **3**
3.1 Harris cổ điển . . . 3
3.2 Hạn chế thứ nhất: không bất biến tỉ lệ . . . 3
3.3 Biến thể A1 — Harris–Laplace . . . 3
3.4 Hạn chế thứ hai: phân bố điểm không đều . . . 4
3.5 Biến thể A2 — Adaptive non-maximal suppression . . . 4
3.6 Hàm phải nộp — Phần A . . . 4
3.7 Kiểm chứng cài đặt — Phần A . . . 5

4 Phần B — Region Growing và SRG **6**
4.1 Region Growing cổ điển . . . 6
4.2 SRG — phát triển đồng thời, sắp theo tương phản . . . 6
4.3 Hai chi tiết cài đặt được chấm điểm . . . 6
4.3.1 Giá trị δ cũ trong hàng đợi . . . 6
4.3.2 Xử lý đồng hạng (ties) . . . 7
4.4 Hàm phải nộp — Phần B . . . 7
4.5 Kiểm chứng cài đặt — Phần B . . . 7

5 Thí nghiệm **8**
5.1 Yêu cầu . . . 8
5.2 Cấu trúc điểm . . . 9
5.3 Trường hợp bị 0 điểm . . . 9

---

1 Mục tiêu

Bài tập gồm **hai phần độc lập**. Mỗi phần: cài đặt một thuật toán kinh điển, cài đặt các biến thể của nó, rồi *thiết kế thí nghiệm* để chỉ ra chúng khác nhau ở đâu và vì sao.

Phần	Thuật toán kinh điển	Biến thể
A (Chương 3)	Harris corner detector	**Hai biến thể độc lập:** A1 — Harris–Laplace A2 — Adaptive non-maximal suppression
B (Chương 4)	Region Growing	Seeded Region Growing (SRG)

2 Quy định chung

2.1 Chính sách sử dụng AI

Áp dụng ở mức từng cell, giống bài tập trước:

- **Phần cài đặt thuật toán — không dùng AI.** Toàn bộ nội dung các cell dưới tiêu đề *"Cài đặt — KHÔNG dùng AI"* phải do bạn tự viết, kể cả việc nhờ AI sửa lỗi hay viết lại.
- **Phần thí nghiệm — được dùng AI** cho việc sinh/chỉnh sửa ảnh, dựng khung thí nghiệm, vẽ biểu đồ. Nhưng *ý tưởng thí nghiệm phải là của bạn*: chọn ảnh nào, quét tham số nào, đo đại lượng gì, kết luận ra sao.
- **Khai báo prompt.** Mỗi cell có dùng AI phải kèm một cell markdown ngay phía trên, chép nguyên văn prompt đã dùng. Dùng AI mà không khai báo ⇒ 0 điểm toàn bài.

2.2 Thư viện

Luôn được phép	Được phép có điều kiện	Không dùng trong phần cài đặt
`numpy`, `matplotlib`, `math`, `time`	`scipy.ndimage.gaussian_filter` — làm mờ Gauss	`cv2.cornerHarris`
	`heapq` — cho SRG	`cv2.goodFeaturesToTrack`
	`cv2.imread`, `cvtColor`, `resize` — nạp và đổi kích thước ảnh	`skimage.feature.corner_*`
	`scipy.ndimage.label` — ở phần phân tích	`skimage.segmentation.*`, `cv2.watershed`

Ở **phần thí nghiệm**, được dùng hàm dựng sẵn để đối chiếu kết quả của mình — và nên làm vậy.

2.3 Nộp bài

Một notebook duy nhất `bt1_<Nhom>.ipynb`, đã chạy *Restart & Run All* và lưu kèm output. Ảnh dùng trong bài đặt trong thư mục `anh/` nộp cùng.

3 Phần A — Harris và hai biến thể

3.1 Harris cổ điển

Ý tưởng gốc [1]: xét một cửa sổ nhỏ quanh điểm $\mathbf{x}$ và hỏi cường độ ảnh thay đổi bao nhiêu khi dịch cửa sổ theo mọi hướng. Xấp xỉ bậc nhất của tổng bình phương sai khác dẫn tới ma trận mô-men bậc hai:

$$M(\mathbf{x};\sigma_I,\sigma_D) \;=\; \sigma_D^2\, G(\sigma_I) * \begin{bmatrix} L_x^2(\mathbf{x};\sigma_D) & L_xL_y(\mathbf{x};\sigma_D) \\ L_xL_y(\mathbf{x};\sigma_D) & L_y^2(\mathbf{x};\sigma_D) \end{bmatrix} \tag{1}$$

với $L_x(\mathbf{x};\sigma_D) = \partial_x[G(\sigma_D) * I]\,(\mathbf{x})$. Hai tham số tỉ lệ có vai trò khác nhau:

- σ_D (*differentiation scale*) — độ mờ trước khi lấy đạo hàm; quyết định ta "nhìn" cấu trúc ở độ mịn nào.
- σ_I (*integration scale*) — bán kính cửa sổ lấy trung bình các tích đạo hàm; quyết định vùng lân cận nào được coi là cùng một cấu trúc.

Quy ước thông dụng: $\sigma_D = s\,\sigma_I$ với $s \approx 0{,}7$.

Đáp ứng góc:

$$R(\mathbf{x}) \;=\; \det M - k\,(\operatorname{tr} M)^2, \qquad k \in [0{,}04;\, 0{,}06] \tag{2}$$

Điểm góc là cực đại địa phương của R vượt ngưỡng. Giá trị $R < 0$ ứng với cạnh chứ không phải góc.

Hệ số σ_D^2 trong (1) là phép chuẩn hóa theo tỉ lệ của Lindeberg [2], làm cho R ở các tỉ lệ khác nhau so sánh được với nhau. Với Harris một tỉ lệ nó chỉ là hằng số nhân; với Harris–Laplace thì nó cần thiết.

3.2 Hạn chế thứ nhất: không bất biến tỉ lệ

Harris bất biến với phép quay, vì det và tr không đổi khi M bị quay. Nhưng nó **không** bất biến với phép đổi tỉ lệ: một góc của hình vuông 20×20 px và cùng góc đó trong hình vuông 80×80 px là hai cấu trúc khác nhau đối với một σ_D cố định — cái thứ hai gần như là "cạnh thẳng" trong cửa sổ đang xét.

3.3 Biến thể A1 — Harris-Laplace

Mikolajczyk và Schmid [3] dựa trên kết quả của Lindeberg: *tỉ lệ đặc trưng của một cấu trúc địa phương được chỉ ra bởi cực trị theo tỉ lệ của đạo hàm đã chuẩn hóa*. Thuật toán có hai giai đoạn.

1. **Xây tháp tỉ lệ.** $\sigma_n = \xi^n \sigma_0$, $n = 0, \dots, N$, thường $\xi \approx 1{,}2$–$1{,}4$. Ở mỗi σ_n: đặt $\sigma_I = \sigma_n$, $\sigma_D = 0{,}7\sigma_n$, tính R theo (1)–(2), lấy cực đại địa phương trong mặt phẳng ảnh và lọc ngưỡng.

2. **Lọc theo trục tỉ lệ.** Giữ điểm $\mathbf{x}$ ở tỉ lệ σ_n khi và chỉ khi Laplacian đã chuẩn hóa

$$|\mathrm{LoG}(\mathbf{x};\sigma_n)| \;=\; \sigma_n^2 \left|L_{xx}(\mathbf{x};\sigma_n) + L_{yy}(\mathbf{x};\sigma_n)\right| \tag{3}$$

đạt cực đại địa phương *theo trục tỉ lệ*:

$$|\mathrm{LoG}(\mathbf{x};\sigma_n)| > |\mathrm{LoG}(\mathbf{x};\sigma_{n-1})| \quad \text{và} \quad |\mathrm{LoG}(\mathbf{x};\sigma_n)| > |\mathrm{LoG}(\mathbf{x};\sigma_{n+1})|$$

và vượt một ngưỡng tối thiểu.

Kết quả: mỗi điểm đặc trưng mang **ba** tọa độ (x, y, σ) thay vì hai.

Bước 2 không phải là lấy $\arg\max_n R(\mathbf{x}; \sigma_n)$. Cực đại của R theo tỉ lệ hầu như luôn tồn tại, còn cực trị LoG thì không: điểm nào không có cực trị LoG theo tỉ lệ sẽ bị loại hoàn toàn. Chính bước loại đó tạo ra tính bất biến tỉ lệ, và đó là nội dung cần được kiểm chứng bằng thí nghiệm.

3.4 Hạn chế thứ hai: phân bố điểm không đều

Nếu lấy N điểm có R lớn nhất, chúng dồn cụm vào vùng tương phản cao — mặt tiền tòa nhà nhiều cửa sổ, vùng có nắng — và bỏ trắng phần còn lại của ảnh. Hạ ngưỡng cũng không giải quyết được: vùng đậm sinh thêm hàng nghìn điểm trước khi vùng nhạt sinh ra điểm đầu tiên.

Hạn chế này độc lập hoàn toàn với hạn chế về tỉ lệ, và xuất hiện ngay cả khi mọi cấu trúc trong ảnh có cùng một tỉ lệ.

3.5 Biến thể A2 — Adaptive non-maximal suppression

Brown, Szeliski và Winder [4] xếp hạng điểm không theo độ mạnh mà theo **bán kính mà tại đó nó thôi là cực đại**:

$$r_i \;=\; \min_{j} \left\| \mathbf{x}_i - \mathbf{x}_j \right\| \qquad \text{với ràng buộc} \qquad f(\mathbf{x}_i) \;<\; c_{\text{robust}} \cdot f(\mathbf{x}_j) \tag{4}$$

trong đó f là đáp ứng góc và $c_{\text{robust}} = 0{,}9$.

Diễn giải: r_i là khoảng cách tới điểm gần nhất *mạnh hơn đáng kể* so với $\mathbf{x}_i$. Điểm mạnh nhất ảnh có $r = \infty$. Sau đó sắp xếp r_i giảm dần và giữ N điểm đầu.

Hệ quả: một điểm yếu nhưng nằm cô lập giữa vùng trống vẫn được giữ, vì không có điểm mạnh nào ở gần nó.

Hệ số $c_{\text{robust}} < 1$ có vai trò riêng. Nếu đặt $c_{\text{robust}} = 1$, hai điểm có đáp ứng gần bằng nhau sẽ triệt tiêu lẫn nhau chỉ vì chênh lệch cỡ sai số dấu phẩy động. Hệ số $0{,}9$ yêu cầu điểm kia phải mạnh hơn khoảng 11% mới được quyền triệt tiêu — và bản thân giá trị này là một trục quét tham số đáng khảo sát.

Trong bài tập này, ANMS chạy trên tập điểm của **Harris cổ điển một tỉ lệ**: nó thay thế bước "lấy N điểm mạnh nhất", và được so sánh trực tiếp với bước đó.

3.6 Hàm phải nộp — Phần A

```
def gaussian_blur(img, sigma):
    """Làm mờ Gauss. Được phép gọi scipy.ndimage.gaussian_filter."""

def structure_tensor(img, sigma_i, sigma_d):
    """Ma trận mô-men bậc hai theo công thức (1), đã nhân sigma_d**2.
    sigma_d: độ mờ trước khi lấy đạo hàm.
    sigma_i: bán kính cửa sổ lấy trung bình các tích đạo hàm.
    -> (Mxx, Mxy, Myy), mỗi phần tử cùng kích thước ảnh.
    """

def harris_response(img, sigma_i, sigma_d, k=0.04):
    """Đáp ứng R theo công thức (2). -> mảng float cùng kích thước ảnh."""

def nms_2d(response, window=3):
    """Cực đại địa phương trong cửa sổ window x window. -> mảng bool."""

def harris_classic(img, sigma_i=2.0, sigma_d=1.4, k=0.04, thresh_rel=0.01):
```

```
    """Harris một tỉ lệ: R -> nms_2d -> ngưỡng tương đối theo max(R).
    -> mảng (N, 3): (row, col, response), sắp xếp giảm dần theo response.
    """

# ---- Biến thể A1 ----------------------------------------------------

def scale_pyramid(sigma0=1.2, xi=1.2, n_levels=10):
    """Danh sách sigma_n = xi^n * sigma0. -> mảng 1D."""

def log_normalized(img, sigma):
    """Laplacian chuẩn hoá theo tỉ lệ: sigma^2 * |Lxx + Lyy|."""

def harris_laplace(img, sigmas, k=0.04, thresh_rel=0.01, log_thresh=0.0):
    """Harris-Laplace (Mikolajczyk & Schmid 2004).
    Bước 1: mỗi sigma -> harris_response -> nms_2d -> ngưỡng.
    Bước 2: giữ điểm chỉ khi |LoG| là cực đại địa phương theo trục tỉ lệ.
    -> mảng (N, 4): (row, col, sigma, response).
    """

# ---- Biến thể A2 ----------------------------------------------------

def anms(points, n_keep, c_robust=0.9):
    """Adaptive non-maximal suppression theo công thức (4).
    points: mảng (N, 3) như đầu ra của harris_classic.
    -> mảng (n_keep, 3), sắp xếp giảm dần theo r.
    """

def top_n(points, n_keep):
    """Đường cơ sở để so với anms: lấy n_keep điểm mạnh nhất.
    -> mảng (n_keep, 3).
    """

# ---- Đo đạc ---------------------------------------------------------

def repeatability(pts_a, pts_b, transform, img_shape, eps=3.0):
    """Tỉ lệ điểm của ảnh A tìm lại được trong ảnh B sau phép biến đổi.
    transform: hàm (row, col) của A -> (row, col) tương ứng ở B.
    Chỉ tính trên các điểm mà ảnh chiếu còn nằm trong B.
    -> số thực trong [0, 1].
    """
```

3.7 Kiểm chứng cài đặt — Phần A

Chạy đủ các kiểm chứng sau, có output, trước khi sang phần thí nghiệm.

1. **Bàn cờ tổng hợp.** `harris_classic` trả về đúng các giao điểm trong của ảnh bàn cờ, sai số ≤ 1 px.
2. **Bất biến phép quay.** Quay ảnh $30°$, chạy lại, so số điểm và vị trí điểm bằng `repeatability`.
3. **Tỉ lệ.** Cùng một hình ở hai tỉ lệ (phóng to $3\times$): tính `repeatability` cho `harris_classic` và cho `harris_laplace`, báo cáo cả hai con số.
4. **Đối chiếu thư viện.** So `harris_classic` với `cv2.cornerHarris`. Không kỳ vọng trùng, nhưng phần lớn điểm của bạn nên nằm trong bán kính 2–3 px của một điểm OpenCV.

5. **Phân bố ANMS.** Với cùng N, so `top_n` và `anms` bằng `grid_occupancy` (hàm cho sẵn): tỉ lệ ô lưới có ít nhất một điểm.

4 Phần B — Region Growing và SRG

4.1 Region Growing cổ điển

Dạng quen thuộc: chọn hạt giống, đẩy các điểm lân cận vào một ngăn xếp, lần lượt lấy ra và kết nạp nếu thỏa mãn tính đồng nhất.

```
khởi tạo: stack = [seed], label[seed] = i, mu = I[seed], n = 1
while stack không rỗng:
    p = stack.pop()
    for q in lân_cận_4(p):
        if q chưa gán nhãn and |I[q] - mu| <= T:
            label[q] = i
            cập nhật mu, n
            stack.push(q)
```

4.2 SRG — phát triển đồng thời, sắp theo tương phản

Adams và Bischof [5] cho **tất cả các vùng phát triển đồng thời**, và để *độ tương phản* quyết định thứ tự kết nạp thay cho thứ tự duyệt.

Khởi tạo n tập hạt giống $A_1, \dots, A_n$, mỗi tập là một pixel hoặc một vùng liên thông. Gọi T là tập các pixel chưa gán nhãn nhưng kề với ít nhất một vùng:

$$T = \Big\{ \mathbf{x} \notin \textstyle\bigcup_i A_i \;:\; N(\mathbf{x}) \cap \bigcup_i A_i \neq \emptyset \Big\}$$

Với mỗi $\mathbf{x} \in T$, gọi $i(\mathbf{x})$ là chỉ số vùng kề và định nghĩa

$$\delta(\mathbf{x}) \;=\; \Big| I(\mathbf{x}) - \operatorname{mean}\{ I(\mathbf{y}) : \mathbf{y} \in A_{i(\mathbf{x})} \} \Big| \tag{5}$$

Nếu $\mathbf{x}$ kề nhiều vùng, lấy $i(\mathbf{x})$ là vùng làm δ nhỏ nhất và đánh dấu $\mathbf{x}$ là pixel biên.

Vòng lặp: lấy $\mathbf{x} \in T$ có δ nhỏ nhất *toàn cục*, kết nạp vào $A_{i(\mathbf{x})}$, cập nhật trung bình vùng, đưa các lân cận mới lộ ra vào T. Lặp tới khi mọi pixel có nhãn.

Ba hệ quả

1. **Phân hoạch đầy đủ**: mọi pixel đều có nhãn.
2. **Không còn ngưỡng đồng nhất.** Toàn bộ độ khó chuyển từ "chọn ngưỡng" sang "đặt hạt giống".
3. **Quyết định chắc chắn nhất diễn ra trước.** Pixel biên mơ hồ được xử lý sau cùng, khi các vùng đã tích lũy đủ thông tin thống kê.

Với hàng đợi ưu tiên, độ phức tạp là $O(n \log n)$ trên n pixel, đủ nhanh cho ảnh 512×512.

4.3 Hai chi tiết cài đặt được chấm điểm

4.3.1 Giá trị δ cũ trong hàng đợi

Mỗi lần kết nạp một pixel, trung bình của vùng đó thay đổi, khiến mọi phần tử đang nằm trong hàng đợi kề với vùng đó có δ không còn đúng. Tính lại toàn bộ thì quá tốn kém. Hai chiến lược:

(a) Vô hiệu hóa lười (lazy)	**(b) Đóng băng theo hạt giống (frozen)**
Khi lấy phần tử ra, tính lại δ; nếu khác giá trị đã lưu thì đẩy lại vào hàng đợi thay vì kết nạp.	δ tính theo giá trị của hạt giống, không theo trung bình vùng. Không bao giờ cũ.
Đúng với định nghĩa gốc. Hàng đợi phình to hơn.	Nhanh và đơn giản. Vùng không thích nghi được với chuyển màu từ từ.

Cài đặt cả hai và so sánh; ảnh có chuyển màu (gradient) mượt là nơi khác biệt lộ rõ nhất.

4.3.2 Xử lý đồng hạng (ties)

Khi nhiều pixel có cùng δ nhỏ nhất — rất hay xảy ra ở vùng phẳng hoặc ảnh 8-bit đã lượng tử hóa — quy tắc phá hòa làm quay lại đúng sự phụ thuộc thứ tự mà SRG muốn loại bỏ. Đây là hạn chế đã được ghi nhận của SRG và là động cơ của thuật toán ISRG [6].

Yêu cầu: làm cho quy tắc phá hòa của bạn tất định (ví dụ theo thứ tự chèn), rồi chỉ ra bằng thí nghiệm rằng thay quy tắc phá hòa vẫn làm đổi kết quả trên ít nhất một ảnh.

4.4 Hàm phải nộp — Phần B

```
def region_growing_classic(img, seeds, T, order="stack"):
    """Region growing cổ điển, một hoặc nhiều hạt giống.
    order: "stack" (LIFO) hoặc "queue" (FIFO).
    -> mảng nhãn int32, -1 cho pixel không thuộc vùng nào.
    """

def srg(img, seeds, connectivity=4, stale="lazy", tie="insertion"):
    """Seeded Region Growing (Adams & Bischof 1994), công thức (5). Dùng heapq.
    stale: "lazy"   -> tính lại delta khi pop, đẩy lại nếu đã cũ
           "frozen" -> delta theo giá trị hạt giống
    tie:   quy tắc phá hoà tất định: "insertion" | "raster" | "random"
    -> (labels, boundary_mask); mọi pixel đều có nhãn >= 0.
    """

def srg_step_frames(img, seeds, n_frames=8, **kwargs):
    """Ảnh nhãn tại các mốc tiến trình (12%, 25%, ...).
    -> danh sách n_frames mảng nhãn.
    """

def seed_sensitivity(img, seeds, segment_fn, n_trials=20, jitter=3):
    """Chạy lại segment_fn với hạt giống bị nhiễu +-jitter px.
    -> bản đồ tần suất: mỗi pixel đổi nhãn bao nhiêu lần trên n_trials.
    Cách định lượng độ ổn định mà không cần ground truth.
    """
```

4.5 Kiểm chứng cài đặt — Phần B

1. **Phân hoạch đầy đủ.** Sau `srg`, khẳng định `(labels >= 0).all()`.
2. **Bảo toàn số vùng.** Số nhãn phân biệt bằng đúng số hạt giống.

3. **Bất biến thứ tự hạt giống.** Chạy `srg` với danh sách hạt giống, rồi với chính danh sách đó đã đảo ngược; so hai kết quả. Đây là tính chất trung tâm của SRG.
4. **Đối chiếu với Region Growing.** Trên cùng một ảnh, so `region_growing_classic` với `order="stack"` và `order="queue"`, rồi so với `srg`. Báo cáo số pixel khác nhau trong từng cặp.
5. **Ảnh phẳng.** Ảnh hằng số với hai hạt giống: mọi δ bằng nhau, nên kết quả phản ánh trực tiếp quy tắc phá hòa của bạn. Mô tả hình dạng thu được.

5 Thí nghiệm

Đây là phần chiếm nhiều điểm nhất và là phần **mở**. Không có ảnh bắt buộc, không có kịch bản bắt buộc.

5.1 Yêu cầu

Với mỗi cặp *kinh điển vs biến thể* — tức ba cặp: A1 vs Harris, A2 vs Harris, SRG vs Region Growing — bạn phải trình bày ba thứ.

1. Bằng chứng khác biệt, trên dữ liệu tự chọn

Tìm ảnh, chỉnh sửa ảnh có sẵn, hoặc tự tổng hợp ảnh bằng code, và có thể kết hợp cả ba. Điều được chấm không phải nguồn ảnh mà là *ảnh đó có làm bật ra khác biệt hay không*, và có *giải thích được vì sao mình chọn nó* hay không.
Kèm ít nhất một đại lượng đo được, không chỉ hình. Ví dụ: tính lặp lại, độ phủ lưới, số pixel đổi nhãn, số vùng, thời gian chạy.

2. Quét tham số không tầm thường

Không phải bảng ba tấm ảnh với ba giá trị k. Một phép quét tham số đạt yêu cầu khi nó cho ra **một đường cong hoặc một điểm chuyển pha**, và bạn nối được hiện tượng đó với cơ chế của thuật toán.
Vài trục đáng quét (danh sách gợi ý, không bắt buộc, khuyến khích tự nghĩ trục khác):

- **A1:** tỉ số ξ giữa các tầng và số tầng — repeatability bão hòa ở đâu? tỉ số $s = \sigma_D/\sigma_I$ ảnh hưởng thế nào tới việc chọn tỉ lệ? `log_thresh` loại bỏ bao nhiêu phần trăm ứng viên?
- **A2:** c_{robust} từ $0{,}5$ tới $1{,}0$ — độ phủ thay đổi ra sao, và có giá trị nào mà hành vi đột ngột đổi không? N thay đổi — độ phủ của `anms` và của `top_n` phân kỳ từ đâu?
- **B:** ngưỡng T của Region Growing — vẽ số vùng hoặc diện tích vùng lớn nhất theo T; có ngưỡng tới hạn nào mà rò rỉ xảy ra không, và nó liên hệ thế nào với tương phản của ảnh? Số lượng và vị trí hạt giống của SRG — dùng `seed_sensitivity`. Liên thông 4 so với 8.

3. Giải thích thay đổi

Nối hiện tượng quan sát được với *bước tính toán cụ thể* đã thay đổi. Ba câu cần trả lời: đại lượng nào được tính khác đi, ở bước nào, và vì sao điều đó dẫn tới đúng hiện tượng đã quan sát.
Không chấp nhận giải thích ở mức "biến thể tốt hơn vì nó bất biến tỉ lệ".

5.2 Cấu trúc điểm

Hạng mục	Phần A	Phần B
Cài đặt thuật toán kinh điển, qua kiểm chứng	8	12
Cài đặt biến thể, qua kiểm chứng	11 (A1) + 11 (A2)	18
Thí nghiệm — bằng chứng khác biệt	6	6
Thí nghiệm — quét tham số không tầm thường	6	6
Thí nghiệm — giải thích thay đổi	5	5
Tổng	**50**	**50**

5.3 Trường hợp bị 0 điểm

1. Dùng AI trong phần cài đặt, hoặc dùng AI ở phần thí nghiệm mà không hề khai báo prompt nguyên văn nào.
2. Gọi hàm phát hiện góc / phân vùng dựng sẵn bên trong các hàm phải nộp.
3. Notebook không chạy được từ đầu đến cuối, hoặc nộp mà không có output.

References

[1] C. Harris and M. Stephens, “A combined corner and edge detector,” in *Proc. 4th Alvey Vision Conf.*, Manchester, UK, 1988, pp. 147–151.

[2] T. Lindeberg, “Feature detection with automatic scale selection,” *Int. J. Comput. Vis.*, vol. 30, no. 2, pp. 79–116, 1998.

[3] K. Mikolajczyk and C. Schmid, “Scale & affine invariant interest point detectors,” *Int. J. Comput. Vis.*, vol. 60, no. 1, pp. 63–86, 2004. doi: 10.1023/B:VISI.0000027790.02288.f2

[4] M. Brown, R. Szeliski, and S. Winder, “Multi-image matching using multi-scale oriented patches,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, San Diego, CA, USA, 2005, vol. 1, pp. 510–517. doi: 10.1109/CVPR.2005.235

[5] R. Adams and L. Bischof, “Seeded region growing,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 6, pp. 641–647, Jun. 1994. doi: 10.1109/34.295913

[6] A. Mehnert and P. Jackway, “An improved seeded region growing algorithm,” *Pattern Recognit. Lett.*, vol. 18, no. 10, pp. 1065–1071, 1997.

[7] J. Shi and C. Tomasi, “Good features to track,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 1994, pp. 593–600.